Studying a Penning trap system using 4-th order Range-Kutta methods

Britt Haanen
(Dated: October 22, 2026)

Particle traps are a fundamental part of particle physics research. In this report we investigate these elegant yet complex systems through numerical simulation using the C++ language. Time evolution of the system is implemented using the 4th-order Range-Kutta (RK4) method, with Forward Euler (FE) method for comparison. For our initial single-particle system we found the particle motion consistent with expectations, and superiority of RK4 compared to FE demonstrated by an error several orders of magnitude smaller. For a two-particle system we studied the effect of Coulomb interactions on particle motion, and found that particle interactions cause a periodic shift in motion. Finally, for a 100-particle system, we studied resonance phenomena and our observations could be linked to ω_z consistent with literature. Overall, this paper presents an efficient C++ implementation of a Penning trap system which was successfully used to study related physics phenomena.

I. INTRODUCTION

Particle traps are a useful tool in physics research, allowing scientists at for example CERN to store and measure properties of (anti)matter particles [1, 2]. Penning traps are a type of particle trap using an electric potential applied by three electrodes in combination with a magnetic field. Penning traps have a rich track record in research, including applications in high energy physics, spectroscopy [3], and antimatter research [1, 2].

Simulation of particle traps are interesting in themselves since they allow one to plot and study the motion of particles in the trap in greater detail. Penning traps provide a nice project for learning to code physics interactions in the trap, since results can be compared against analytical solutions for a single particle as benchmark, before continuing on to more complex multi-particle systems.

The aim of this report was both to create an object-oriented code in C++ to simulate a Penning trap, and to study the motions of one or two particles in the trap and how they are affected by Coulomb interactions. Finally, the code would also be used to investigate resonances in a system of 100 particles.

In Section II we describe the mathematical and physical background of the Penning trap. We will formulate an algorithm for numerically solving the system using the fourth-order Runge-Kutta (RK4) method. Also, we will describe the object-oriented C++ implementation of the simulation and the steps undertaken as validation. A selection of results from running the validated system with various settings will be presented in Section III. Finally, in Section IV we provide a short summary and outlook. All code developed for this report is available via a GitHub repository.[1]

[1] https://github.com/BrittHaanen15/Project3-final-version.git

II. METHODS

A. Mathematical formulation

A particle in a Penning trap is subject to the Lorentz force,

$$\vec{\boldsymbol{F}} = q\vec{\boldsymbol{E}} + q\vec{\boldsymbol{v}} \times \vec{\boldsymbol{B}} \tag{1}$$

where $\vec{\boldsymbol{F}}$ is the Lorentz force resulting from the fields that the particle finds itself in (electric field $\vec{\boldsymbol{E}}$ and magnetic field $\vec{\boldsymbol{B}}$), the particle velocity $\vec{\boldsymbol{v}}$, and the particle charge q.

For the particular Penning trap problem we are solving in this report, the fields have been defined as

$$\vec{\boldsymbol{E}} = -\nabla V = -\nabla \frac{V_0}{2d^2}(2z^2 - x^2 - y^2)$$
$$\vec{\boldsymbol{B}} = B_0 \hat{e}_z$$

where (x, y, z) are the coordinates of the particle, V_0 is the potential applied to the electrodes, and d is the characteristic dimension representing the length scale between the electrodes. Finally, B_0 is the magnetic field strength.

Applying Newton's second law $m\ddot{\boldsymbol{r}} = \sum \boldsymbol{F_i}$ where we set Equation (1) as $\boldsymbol{F}$, and switching to dot notation for derivatives, we obtain the vector equation

$$m\begin{pmatrix}\ddot{x}\\ \ddot{y}\\ \ddot{z}\end{pmatrix} = q(-\frac{V_0}{d^2}\begin{pmatrix}-x\\ -y\\ 2z\end{pmatrix} + \begin{pmatrix}\dot{x}\\ \dot{y}\\ \dot{z}\end{pmatrix} \times \begin{pmatrix}0\\ 0\\ B_0\end{pmatrix})$$
$$\frac{m}{q}\begin{pmatrix}\ddot{x}\\ \ddot{y}\\ \ddot{z}\end{pmatrix} = \frac{V_0}{d^2}\begin{pmatrix}x\\ y\\ -2z\end{pmatrix} + \begin{pmatrix}B_0\dot{y}\\ -B_0\dot{x}\\ 0\end{pmatrix}$$

with all variables defined as before. This allows us to write a set of equations for each component:

$$\ddot{x} = \frac{V_0 q}{md^2}x + \frac{qB_0}{m}\dot{y}$$
$$\ddot{y} = \frac{V_0 q}{md^2}y - \frac{qB_0}{m}\dot{x}$$
$$\ddot{z} = -\frac{2V_0 q}{md^2}z$$

If we introduce $\omega_0 \equiv \frac{qB_0}{m}$ and $\omega_z^2 \equiv \frac{2qV_0}{md^2}$ this simplifies to

$$\ddot{x} - \omega_0\dot{y} - \frac{1}{2}\omega_z^2 x = 0 \tag{2}$$
$$\ddot{y} + \omega_0\dot{x} - \frac{1}{2}\omega_z^2 y = 0 \tag{3}$$
$$\ddot{z} + \omega_z^2 z = 0 \tag{4}$$

Equations (2) and (3) are coupled. By introducing the complex function $f(t) = x(t) + iy(t)$ we can derive this to one function. The defined function $f(t)$ extends to the second derivative so that $\ddot{f} = \ddot{x} + i\ddot{y}$. Thus, we obtain

$$\begin{aligned}\ddot{f} &= \omega_0\dot{y} + \frac{1}{2}\omega_z^2 x + i(\omega_0\dot{x} + \frac{1}{2}\omega_z^2 y) \\ &= \omega_0(\dot{y} - i\dot{x}) + \frac{1}{2}\omega_z^2(x + iy)\end{aligned}$$

Using the definition $f(t) = x(t) + iy(t)$, and by rearranging $\dot{f} = \dot{x} + i\dot{y}$ to obtain $-i\dot{f} = \dot{y} - i\dot{x}$, we can simplify this expression into

$$\ddot{f} + i\omega_0\dot{f} - \frac{1}{2}\omega_z^2 f = 0 \tag{5}$$

Equation (5) has the general solution

$$f(t) = A_+ e^{-i(\omega_+ t + \phi_+)} + A_- e^{-i(\omega_- t + \phi_-)} \tag{6}$$

[4], thus presenting the general solution to Equations (2) and (3). The constraints and bounds of this solution are discussed in more detail in the Appendix. Equation (4) has the general solution

$$z(t) = c_2 \sin(\omega_z t) + c_1 \cos(\omega_z t) \tag{7}$$

which was found with the assistance of Wolfram Alpha [5].

B. Setup of the numerical simulation

The project aim was to solve the described system numerically using a dedicated object-oriented code in C++.

1. *Particle class*

A class 'Particle' was written for storing the mass, charge, location and velocity of a particle. This class was utilised in the other code files to perform a simulation.

2. *Penning Trap class*

A dedicated class 'Penning Trap' formed the heart of the simulation work. This class held variables for storing the trap parameters (such as V, B_0 etc.) and a 'container' for tracking all particles (initialised as instances of class Particle) in the trap. The class contained two member functions for evolving the system forward in time, taking the time step dt as an argument.

Both 'evolution' member functions called on other member functions to evaluate the forces on each single particle. The force on a particle from the trap potential and magnetic field was evaluated according to the Lorentz force (Equation (1)). This used the potential of the trap, which was coded as

$$V_0 \to V_0(1 + f\cos(\omega_V t))$$

allowing the option of applying a potential varying with time according to the amplitude f and the frequency ω_V. Amplitude and frequency of the potential were taken as arguments when initialising the class. Both were defaulted to zero to create a static potential if no amplitude and frequency arguments were passed. Calculating the Coulomb force from other trapped particles was optional, and taken as argument at trap initialisation. If turned on, the electric field from other particles was calculated according to Coulomb's law

$$\boldsymbol{E} = k_e \sum_{j=1}^{n} q_j \frac{\boldsymbol{r} - \boldsymbol{r_j}}{|\boldsymbol{r} - \boldsymbol{r_j}|^3}$$

And the Coulomb force was summed with the Lorentz force to calculate the total force on the particle. If turned off, the Lorentz force was taken as the total force.

As for the member functions for evolving the system, the first of these 'evolution' functions used the Forward Euler (FE) algorithm, serving mainly as a starting point and 'sanity check' for the second function. FE was implemented as described in Algorithm 1.

The second 'evolution' function implemented the Runge-Kutta (RK4) method to evolve the system forward in time. This was the main algorithm used. The algorithm was coded as shown in Algorithm 2.

Finally, the PenningTrap class also contained some member functions for printing member particles to terminal as a sanity check, for checking how many particles were still in the trap, and a function for writing time and particle parameters to a .csv file.

Algorithm 1 Forward Euler method

procedure ONE FORWARD EULER STEP(dt, P)
  for dt **do**
    $k_r \leftarrow dt * v$ ▷ Compute k for position
    Compute F for P ▷ P is particle collection
    $k_v \leftarrow dt * \frac{F}{m}$ ▷ Compute k for velocity
    $P \leftarrow r + k_r, v + k_v$ ▷ Update P using k values

Algorithm 2 Runge-Kutta method

procedure ONE RK4 STEP(dt, P)
  for dt **do**
    $C \leftarrow P$ ▷ Copy particle collection P
    $k_{1r} \leftarrow dt * v_C$ ▷ Compute k1 for position
    Compute F for C ▷ Compute Force
    $k_{1v} \leftarrow dt * (\frac{F}{m})_C$ ▷ Compute k1 for velocity
    $P \leftarrow r_C + 0.5 * k_{1r}, v_C + 0.5 * k_{1v}$ ▷ Update P using k values, C remains unchanged

    $k_{2r} \leftarrow dt * v_C$ ▷ Compute k2 for position
    Compute F for P ▷ Compute Force
    $k_{2v} \leftarrow dt * (\frac{F}{m})_P$ ▷ Compute k2 for velocity
    $P \leftarrow r_C + 0.5 * k_{2r}, v_C + 0.5 * k_{2v}$ ▷ Update P using k values

    $k_{3r} \leftarrow dt * v_C$ ▷ Compute k3 for position
    Compute F for P ▷ Compute Force
    $k_{3v} \leftarrow dt * (\frac{F}{m})_P$ ▷ Compute k3 for velocity
    $P \leftarrow r_C + k_{3r}, v_C + k_{3v}$ ▷ Update P using k values

    $k_{4r} \leftarrow dt * v_C$ ▷ Compute k4 for position
    Compute F for P ▷ Compute Force
    $k_{4v} \leftarrow dt * (\frac{F}{m})_P$ ▷ Compute k4 for velocity
    $P \leftarrow r_C + \frac{1}{6}(k_{1r} + 2k_{2r} + 2k_{3r} + k_{4r})$ ▷ Update to position after timestep using a combination of k values
    $P \leftarrow v_C + \frac{1}{6}(k_{1v} + 2k_{2v} + 2k_{3v} + k_{4v})$ ▷ Update to velocity after timestep using a combination of k values

C. Verification and benchmarking

1. Unit testing

As part of code development, unit tests were written to test the correct initialisation of the Particle and PenningTrap classes. These code files involved a simple initialisation and setting of parameters, and then printing the member variables of the class to the terminal to verify correct initialisation.

The PenningTrap class functions for evaluating the external and particle force fields were evaluated in a unit test using a static field. As part of this unit test two particles were initialised using arbitrary locations and velocities. The resulting forces were computed and printed to terminal for comparison against hand calculations. This confirmed the correctness of the calculation of the Lorentz force, the Coulomb force, and the summed total force.

The FE method was unit tested by running one iteration of the algorithm using the particles as initialised for the previous unit test. Hand calculations were expanded with one FE step to verify against the computed result. This confirmed that the FE implementation was correct.

The RK4 method was harder to fully unit test, as the calculation of all k's is done on updated particle positions throughout the time step. As an alternative, the first set of k values were printed to terminal and verified against hand calculations, which was still feasible. The output from the full timestep was compared against the FE result as we reasoned that this should be slightly different, but not by orders of magnitude. This way, the RK4 implementation was verified, though not in a fully foolproof way. We relied on comparison of a single particle simulation against the analytical result to fully conclude correctness of the RK4 implementation.

2. The analytical benchmark

The analytical solution was provided by A. Kvellestad [4]. For a set of initial conditions:

$$\begin{aligned} x(0) = x_0, \dot{x}(0) = 0 \\ y(0) = 0, \dot{y}(0) = v_0 \\ z(0) = z_0, \dot{z}(0) = 0 \end{aligned}$$

we get the analytical solution

$$z(t) = z_0 \cos(\omega_z t) \tag{8}$$

for the z plane, while Equation 6 becomes a specific solution with

$$\begin{aligned} A_+ &= \frac{v_0 + \omega_- x_0}{\omega_+ - \omega_-} \\ A_- &= \frac{v_0 + \omega + -x_0}{\omega_+ - \omega_-} \\ \phi_+ &= \pi \\ \phi_- &= 0 \end{aligned}$$

We initialised a one-particle simulation using the settings above. We also hard-coded the given expressions in Python using the parameters that were set for our system. The expressions were evaluated for the same timesteps etc. as used for the numerical simulation of a single particle to generate a set of x,y,z coordinates for each time step. The outputs from the numerical simulation for a single particle were imported in the same script and compared against the analytical solution. The outputs from this test served both as a project result (further discussed in Section III) and as a verification of the correctness of the numerical simulation using RK4 method before moving on to multi-particle systems.

Algorithm 3 Simulation logic for exploring resonance

```
procedure EXPLORE RESONANCE(dt, P)
    for f= 0.1, 0.4, 0.7 do
        for ω_V in range(0.2, 2.5) with step 0.2 do
            Initialise PenningTrap(f, ω_z, n_particles = 100, interactions=False)
            while time< 500μs do
                Run RK4
            Record f, ω_V and particles in trap
```

D. Simulations

The parameter settings for the trap and the particles are described in Appendix B.

1. *Single particle*

A single particle was simulated on the time interval $[0, 50]\mu$s using different settings for the number of steps n in the interval. The RK4 and FE method were both used for steps $n = 4000, 8000, 16000, 32000$. The particle positions at each time step were written to .csv files and imported in Python for visualisation. We also produced plots to compare the simulation results to the analytical solution using the relative error.

2. *Two particles*

Two particles were simulated using RK4 and $n = 32000$, both with and without particle interactions. The particle positions and velocities at each time step were written to .csv files. Python plotting was used to analyse the influence of Coulomb interactions on the particle trajectories.

3. *100 particles*

Finally, 100 particles were simulated using random initialisation of particle positions and velocities. The PenningTrap was initialised with a dynamic potential. The aim of the simulation was to explore resonance phenomena, so the simulation was run of different amplitudes f and frequencies ω_V. Each simulation was evolved for 500 μs using RK4, which was implemented using a dedicated `FullRK4` function written in utils. No outputs were written throughout, only the amount of particles inside the trap was recorded at the end of each run, before switching to a new frequency. The implemented logic for this simulation is shown in Algorithm 3. One of the observed resonance frequencies was explored in more detail by simulating it over a more finely spaced frequency grid, both with and without particle interactions.

E. Tools

We used the Python libraries `Numpy` [6] and `pandas` [7] to read the simulation output and handle it in Python. `matplotlib` [8] was used to produce all illustrations in this report.

The simulation code in C++ makes liberal use of C++ standard libraries. In addition we used `Eigen` [9] to handle all vector quantities (such as particle position and velocity) and associated computations.

No LLM's were used to generate either code or text. GPT UiO was used in one instance: debugging the analytical solution in Python, in order to identify if the mistake was in the coded analytical solution or the C++ code. It identified a forgotten square root for ω_z^2. A note has been left in the code to show the LLM use here.

III. RESULTS AND DISCUSSION

A. Single particle system

First, we simulated a single particle system with the settings as detailed in Appendix B. The simulation was run for different numbers of discretisation steps n, with two different solver methods, RK4 and FE.

As a first result and sanity check, we examined the motion in the z direction $z(t)$, which is plotted in Figure 1. With our setting of ω_z and a static V_0 we would expect the particle to remain confined in the trap and exhibit some periodic motion. This is confirmed by the analytical solution (Equation (8)) which shows that we would expect a periodic motion in the z plane with an amplitude $z_0 = 20$ and a period $\frac{2\pi}{\omega_z}$ which for our parameters corresponds to a value of roughly 9. This matches what we observe in Figure 1.

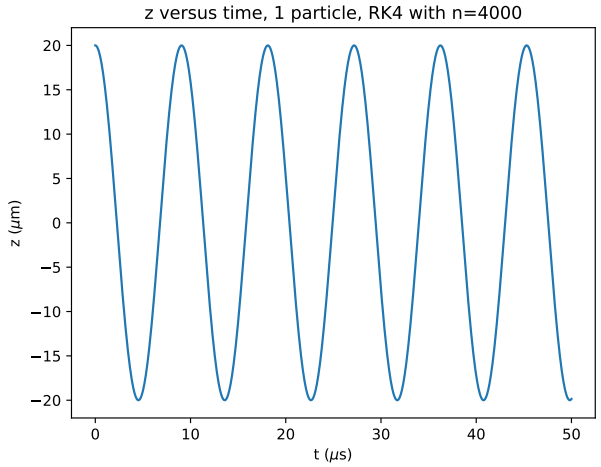


FIG. 1. Position in the z plane as function of time for a single particle system, solved with RK4 with $n = 4000$. The behaviour is consistent with the analytical expectations.

The particle coordinates from the simulation are compared against those obtained via analytical methods using the relative error in position r so that relative error $= \frac{|r_{i,analytical} - r_{i,numerical}|}{|r_{i,analytical}|}$. The results are plotted for different choices of n in Figure 2 for RK4, and in Figure 3 for FE. Both plots exhibit the relative error in position for each time step. FE is a first-order method (i.e. global error $\mathcal{O}(h)$, with local error $\mathcal{O}(h^2)$. RK4 on the other hand, is fourth-order and therefore has local error $\mathcal{O}(h^5)$ and global error $\mathcal{O}(h^4)$ [10]. As a result, we would expect the RK4 method to obtain errors an order of magnitude 10^4 smaller than the FE method. In addition, we would expect the error of RK4 to reduce more sharply with a smaller timestep (thus higher n).

The expected results appear to be borne out in part by Figures 2 and 3 as comparing the two clearly shows the different orders of magnitude of the error. FE appears to exhibit a 'snowball' effect, where the error in position accumulates as timestep increases. This is mitigated somewhat by reducing the step size, leading to the expected reduction in error. What suprised us was the behaviour of the RK4 error, which appears oscillatory rather than accumulating. We were also surprised by the growing sharp 'spikes' of error as n increases, as one would typically assume a lower error for higher n. It is likely that the dips in error for higher n are also more severe, leading to a reduced global error as expected.

B. Two particle system

The two particle system was initialised according to the settings in Appendix B. The main aim of this simulation was to explore how the particle trajectories would be affected by the particle-particle interactions through the Coulomb force. Therefore the simulation was run for both settings and the resulting particle trajectory in the xy plane is plotted in Figure 4. The particle trajectories that we observe in the Figure are consistent with those expected for particles in a Penning trap [p.54, [11]] and as described by the analytical solution which is a sum of two periodic functions. We see that turning on the Coulomb interactions changes the particle trajectories slightly. We speculate that this is due to the added factor of the Coulomb force in the total force acting on the particle: this results in equations of motion with a slightly different periodicity ω. We also plotted the phase spaces (x, v_x) (Figure 5) and (z, v_z) (Figure 6) with and without interactions. For easier interpretation, only particle 1 is shown here. The plots for particle 2 are available in the Github repository. For a particle that is in a stable periodic motion (where energy is conserved) we expect closed ellipses in a phase space plot. Spirals indicate instability, where inward spirals indicate loss of energy and outward spirals indicate gaining energy [12]. This latter would be problematic in a Penning Trap as this would cause the particles to eventually escape. Looking at Figure 5 we can see a shift in the phase space when particle interactions are turned on. This is consistent with the earlier discussed shift in periodicity of the motion. Both scenarios exhibit spiraling. For the (x, v_x) phase space the spiral appears to be going outward with particle interactions, which might indicate gaining energy and the possibility of eventual escape from the trap. Figure 6 shows perhaps a bit more clearly that the particles are in a very stable motion when particle interactions are not accounted for, as indicated by the closed ellipse. When particle interactions are turned on, the phase space plot collapses from the stable ellipse. However, when we examine the endpoints for both scenarios (as indicated by the stars) we see that the endpoints are similar. This is an indicator that the trap with the current settings may be stable even with Coulomb interactions between the particles.

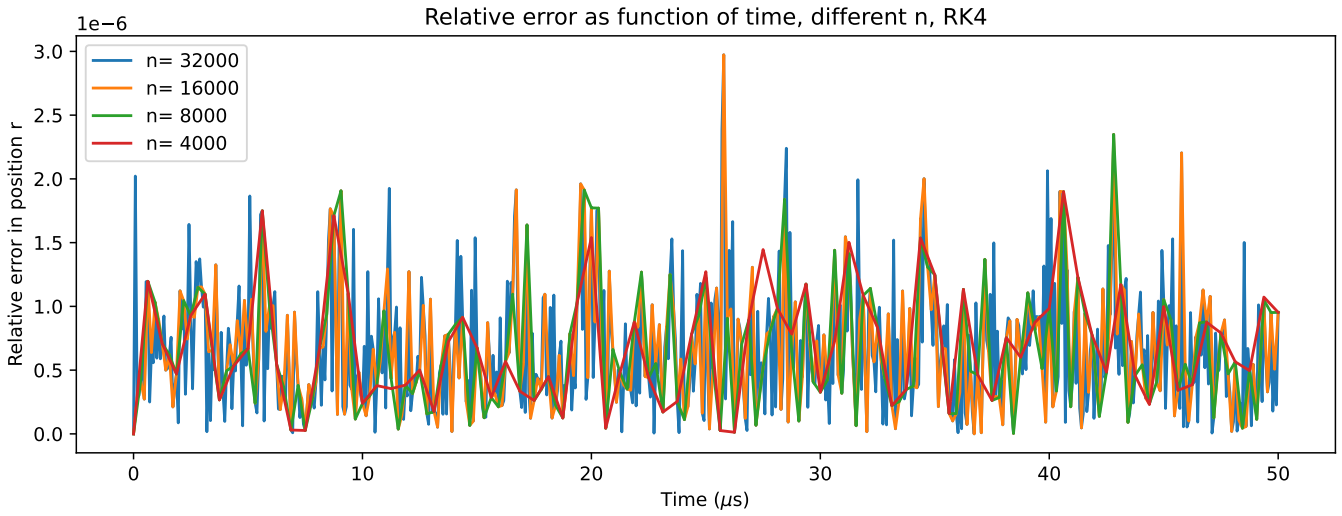


FIG. 2. Evolution of relative error for each timestep for RK4 solver with different discretisations. Note the small error and the increased error spikes for smaller discretisations.

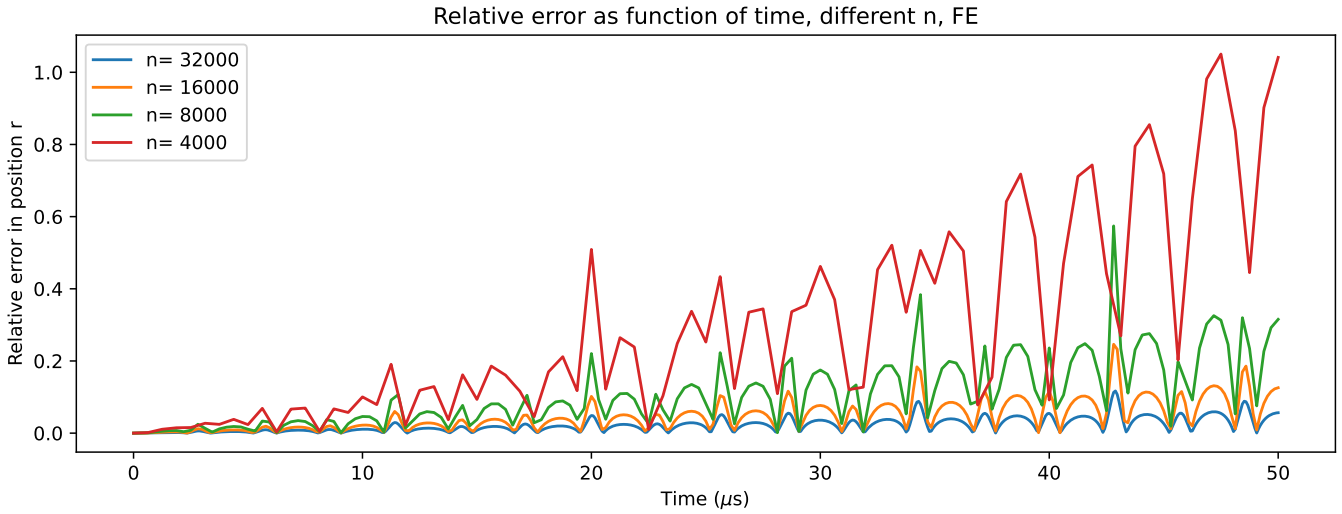


FIG. 3. Evolution of relative error for each timestep for FE solver with different discretisations. Note the much larger scale of the error as compared to RK4.

C. 100 particle system

Finally, we simulated a 100 particle system with a dynamic electric potential to explore resonance effects in our Penning trap. We scanned over a frequency range of $\omega_V = 0.2 - 2.5$ MHz for amplitudes of $f = 0.1, 0.4, 0.7$. Our result is presented as a plot of particles left in the trap after 500μs as a function of the frequency. The results are plotted in Figure 7, where we clearly see 'dips' corresponding to all particles escaping the trap at certain frequencies. These are the resonance frequencies, and the plot looks as expected from other sources [p. 57, [11]]. The plot should show resonance at frequencies ω_+, ω_-, and $n * \omega_z$ [11]. For our trap settings, this would correspond to $2.30, 0.10,$ and $n{*}0.69$ MHz, respectively. We see the multiples of ω_z show up clearly in Figure 7, though ω_+ and ω_- are not as apparent. At these frequencies, the electric field 'drives the undamped harmonic oscillator for z(t) on resonance,eventually leading to a linear growth of the amplitude of z(t)' [13], leading to the particles escaping the trap.

We see that for a greater amplitude the frequency ranges for resonance broaden. We speculate that this is because the field is stronger at greater amplitude, causing a larger effect. For $f = 0.7$, we see a resonance appear at around 0.4 MHz that was not observable at smaller amplitudes.

To conclude our exploration of resonance phenomena,

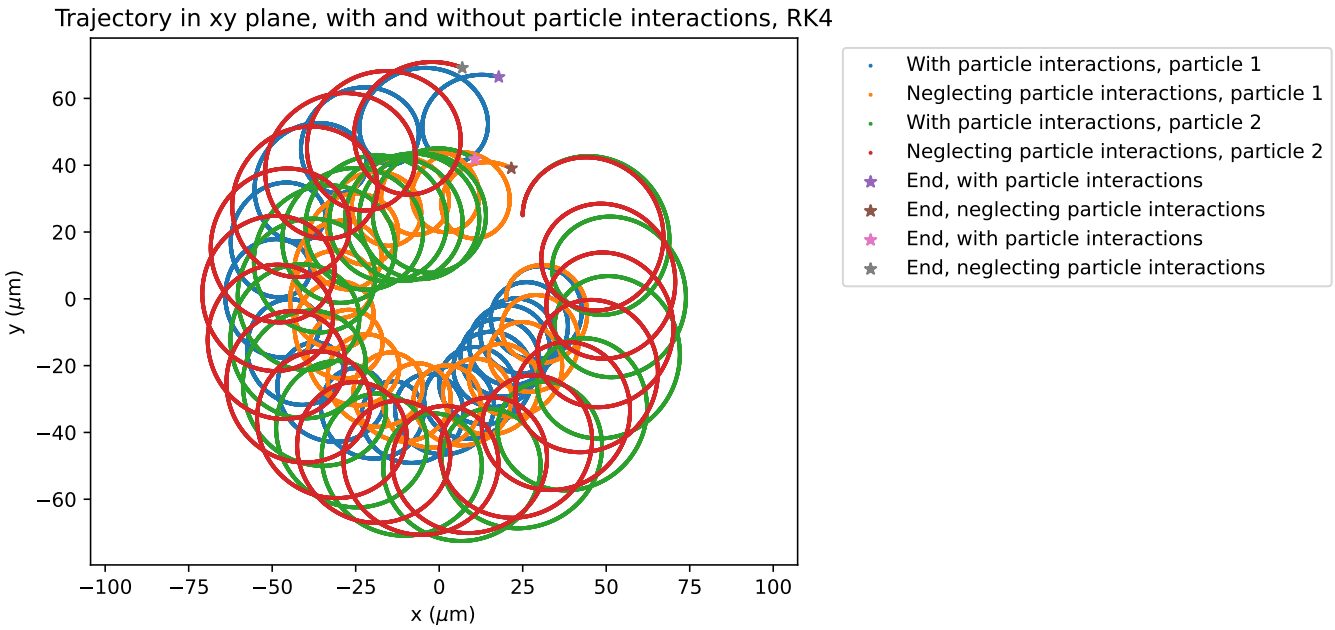


FIG. 4. Motion in the xy plane for a two-particle system, both with and without Coulomb interactions. Solved with RK4 method with $n = 32000$. The particle motion is consistent with the sum of two periodic functions.

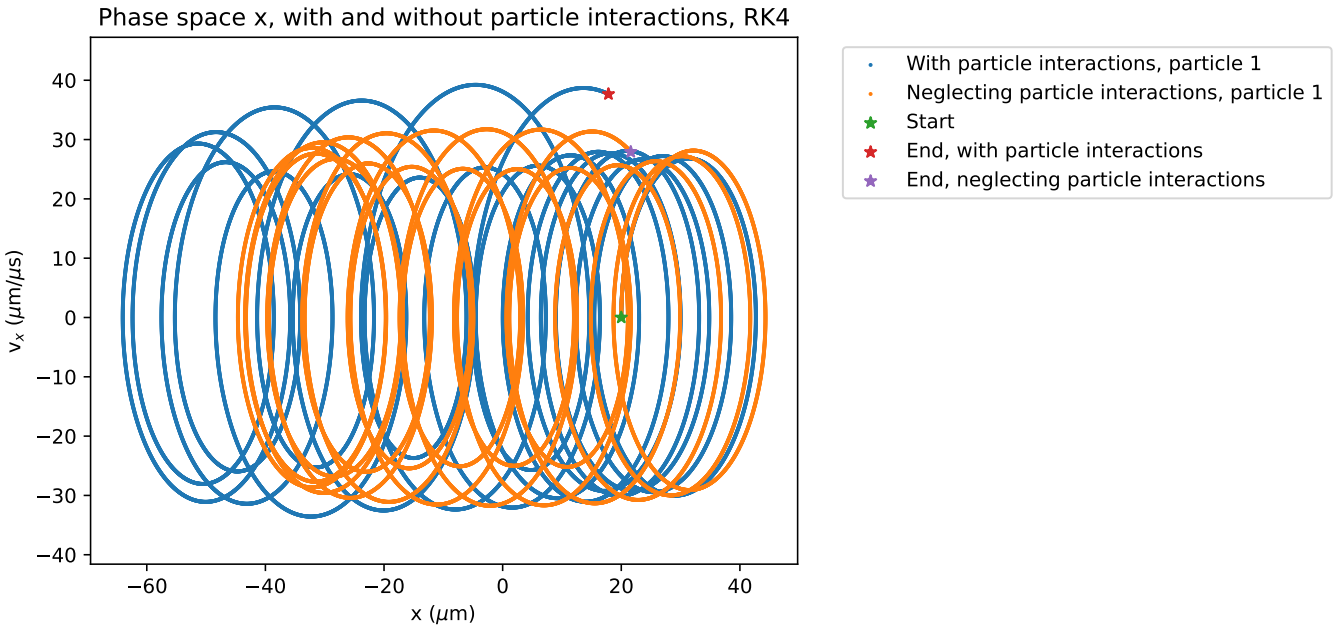


FIG. 5. Phase space (x, v_x) for particle 1 in a two-particle system. Note how the endpoint is shifted outward when particle interactions are turned on.

we searched a more finely spaced frequency grid for one of the resonance frequencies and simulated with particle interactions on and off to see how this affected resonance. We opted for the resonance around 0.70 MHz in Figure 7 using amplitude 0.7, since this instance seemed well-pronounced enough to study in detail but still relatively small compared to the resonance around 1.4 MHz, making it feasible to study in a finely spaced grid. We searched frequencies 0.55-0.75 MHz in steps of 0.002. For both particle interaction settings we ran the simulation five times with different random seeds (1-5) to reduce statistical uncertainty. The results in Figure 8 show the average fraction of particles left in the trap as a function of frequency for both particle interaction settings. We see

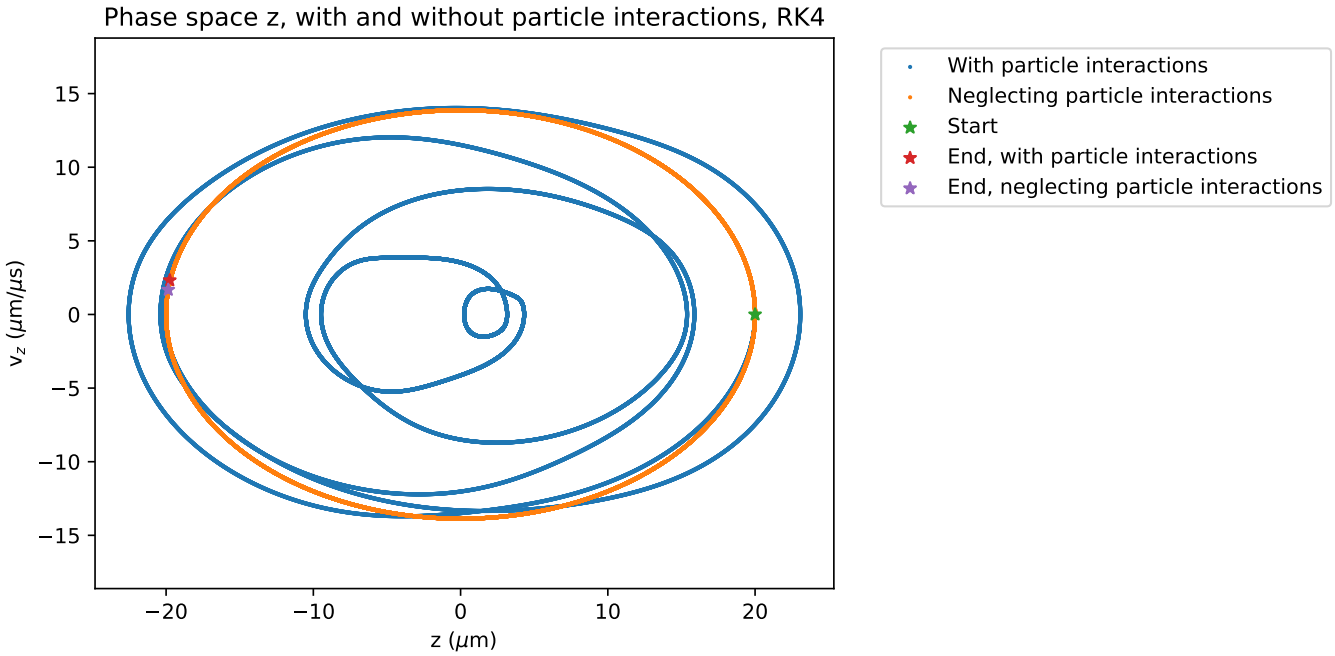


FIG. 6. Phase space (z, v_z) for particle 1 in a two-particle system. Without particle interactions the neat closed ellipse shows a stable system. With particle interactions the trajectory is more complicated, but perhaps still stable considering the same endpoint.

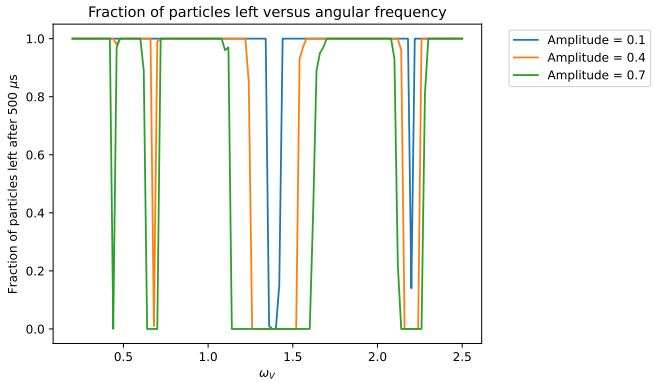


FIG. 7. Particles left in the trap after 500 μs as function of applied frequency in the electric potential, for different amplitudes. Note the resonance phenomena, denoted by the 'dips' where particles escape from the trap, at multiples of ω_z consistent with literature [11].

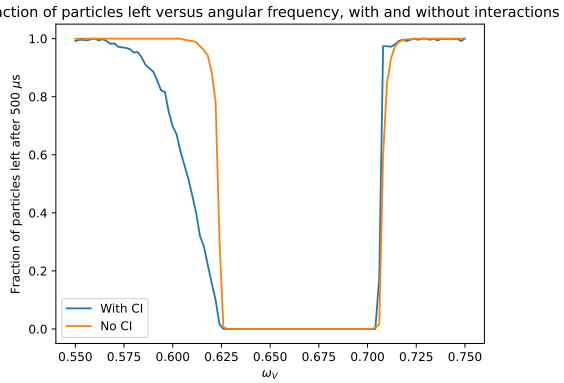


FIG. 8. Particles left in the trap after 500 μs as function of applied frequency in the electric potential (amplitude 0.7), with and without particle interactions. Particle interactions broaden the frequency range where particle escape occurs, and slightly shift the range where all particles escape to the left.

that particle interactions broadens the frequency range for which we observe particle escape from the trap. The frequency range for which we observe total resonance, i.e. all particles escaping the trap, appears minimally shifted to the left. However, this may also be an artefact of the relatively low particle number that was simulated.

IV. CONCLUSION

In this project we created an object-oriented C++ code to simulate a Penning trap system. The system was validated through some unit tests and comparison to hand calculations before it was applied to simulations.

Initially we studied a single particle, and examined its motion as compared to analytical solutions. This also allowed us to study the effect of using fourth-order Runge-

Kutta as compared to Forward Euler to evolve the simulation forward in time. We concluded that though the error in FE can be mitigated by smaller step size, RK4 performs superiorly by solving the system with an error that is several orders of magnitude smaller than FE. Moving on to a two particle system, we examined the effect of Coulomb interactions on the particle motion using both the particle trajectories and phase space plots. We concluded that Coulomb interactions affect, but do not necessarily destabilise, the particle trajectories. Finally, we used a 100 particle simulation with a time-variant electric potential to explore resonance phenomena in our system. We identified resonance frequencies where particles escape from the trap, and explored one of these in more detail both with and without particle interactions. We observed resonance phenomena at multiples of ω_z which was consistent with literature. We also observed broadening of the frequency range for which particles escape from the trap if particle interactions are turned on.

In future, interesting areas to research further would be to study resonance phenomena in more detail, and perhaps link these more closely to the trap and particle properties. Now, we did not observe resonance at ω_+ and ω_-. One could also consider different algorithms for evolving the system, such as Leapfrog or Verlet [10]. Also, the presented solution is from a novice to C++, so we would be especially interested in how to optimise the code to be more computationally efficient. Finally, regarding the previously described applications in research of Penning traps, one could consider exploring how well the current simulation allows itself to be adapted to simulate phenomena in these fields.

Overall, this paper presents a validated and efficient C++ code to study a Penning trap system. The code was successfully used to explore physics phenomena related to this type of particle trap.

[1] CERN. The base experiment at cern compares protons and antiprotons with high precision. *CERN Library: At CERN*, 2015. Retrieved 21/10/2026 from https://home.cern/the-base-experiment-at-cern-compares-protons-and-antiprotons-with-high-precision/.

[2] Rory Harris. Alpha measures tiny energy gap in antimatter with improved precision. *CERN Library: Physics*, 2026. Retrieved 21/10/2026 from https://home.cern/alpha-measures-tiny-energy-gap-in-antimatter-with-improved-precision/.

[3] Richard Thompson. Penning traps - lecture 1. Online version of a presentation given by the author at Imperial College London. Retrieved 21/10/2026 from https://indico.cern.ch/event/596966/sessions/217664/attachments/1580468/2497296/Les_Houches_2018_Penning_L1.pdf.

[4] A. Kvellestad. Project 3 description for FYS3150.

[5] Wolfram Alpha LLC. Wolfram—Alpha. https://wolframalpha.com. Accessed: 2/10/2026.

[6] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.

[7] Wes McKinney. Data structures for statistical computing in python. In Stéfan van der Walt and Jarrod Millman, editors, *Proceedings of the 9th Python in Science Conference*, pages 56 – 61, 2010.

[8] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[9] Gaël Guennebaud, Benoît Jacob, et al. Eigen. https://libeigen.gitlab.io, 2010.

[10] A. Kvellestad. Lecture notes for FYS3150.

[11] Fouad G. Major, Viorica N. Gheorghe, and Günther Werth. *Charged Particle Traps: Physics and Techniques of Charged Particle Field Confinement*, volume 37 of *Springer Series on Atomic, Optical, and Plasma Physics*, chapter 3. Springer Science & Business Media, Berlin, Heidelberg, 2005.

[12] Wikepedia Foundation. Phase space. Retrieved on 21/10/2026 from https://en.wikipedia.org/wiki/Phase_space.

[13] Jochen Ketter, Tommi Eronen, Martin Höcker, Sebastian Streubel, and Klaus Blaum. First-order perturbative calculation of the frequency-shifts caused by static cylindrically-symmetric electric and magnetic imperfections of a penning trap. *International Journal of Mass Spectrometry*, 358:1–16, 2014.

Appendix A: Constraints and bounds on the solution for the xy plane

Recalling the general solution to the equations of motion in the xy plane, (Equation (5))

$$f(t) = A_+ e^{-i(\omega_+ t + \phi_+)} + A_- e^{-i(\omega_- t + \phi_-)}$$

We wish to examine the constraints on ω_0 and ω_z to obtain a bounded solution so that $|f(t)| < \infty$ as $t \to \infty$. These constraints were derived with the aid of Chapter 3 in *Charged particle traps: Physics and Techniques of Charged Particle Field Confinement* [11].

Looking at the form of the equation, we see that $|f(t)| \to \infty$ in case that the exponent $\to \infty$. For this to happen, the total exponent must yield a real number. Thus, the i must cancel out by the production of another i, which would stem from the definition of $\omega_\pm$ [4]

$$\omega_\pm = \frac{\omega_0 \pm \sqrt{\omega_0^2 - 2\omega_z^2}}{2}$$

To produce an imaginary number here and yield a total exponent that is real, the square root has to contain a negative number ('In order that the motion be bounded, the root of [...] must be real'[11]). Thus, we can infer

$$\begin{aligned} \omega_0^2 - 2\omega_z^2 &< 0 \\ \omega_0^2 &< 2\omega_z^2 \end{aligned}$$

as our constraint to obtain a bounded solution. Using our previous definitions of $\omega_0 \equiv \frac{qB_0}{m}$ and $\omega_z^2 \equiv \frac{2qV_0}{md^2}$ we can relate this constraint directly to the parameters of our Penning trap and our particle:

$$\begin{aligned} \frac{q^2 B_0^2}{m^2} &> 2\frac{2qV_0}{md^2} \\ \frac{q^2 B_0^2}{m} &> 4\frac{V_0}{d^2} \\ \frac{d^2 B_0^2}{V_0} &> 4\frac{m}{q} \end{aligned}$$

Here we have the trap parameters (magnetic field strength B_0, characteristic dimension d, and electrode potential V_0) on the left hand side and the particle parameters (charge q and mass m) on the right hand side.

As for the bounds on the solution, we use the given definition that $x = \mathrm{Re}f(t)$ and $y = \mathrm{Im}f(t)$. We can relate these two components to the total radius in the xy plane using $\sqrt{x^2 + y^2}$ [11]. Calculating the squares separately, we obtain

$$\begin{aligned} x^2 &= A_+^2 \cos^2(F_+) + 2A_+A_- \cos(F_+)\cos(F_-) + A_-^2 \cos^2(F_-) \\ y^2 &= A_+^2 \sin^2(F_+) + 2A_+A_- \sin(F_+)\sin(F_-) + A_-^2 \sin^2(F_-) \end{aligned}$$

Where $F_+ = \omega_+ t + \phi_+$ and $F_- = \omega_- t + \phi_-$. Summing these two equations, we can use $\sin^2(x) + \cos^2(x) = 1$ to simplify the first and third terms in the equation to A_+^2 and A_-^2, respectively. For the middle term, we can use the identity $\cos(x)\cos(y) + \sin(x)\sin(y) = \cos(x - y)$ to simplify to $2A_+A_-\cos(F_+ - F_-)$. Thus, we find the radius in the xy plane to be

$$R = \sqrt{A_+^2 + A_-^2 + 2A_+A_-\cos(F_+ - F_-)} \tag{A1}$$

This equation will be maximised for $\cos(F_+ - F_-) = 1$, giving us the maximum

$$R = \sqrt{A_+^2 + A_-^2 + 2A_+A_-} = \sqrt{(A_+ + A_-)^2} = |A_+ + A_-|$$

Similarly, Equation (A1) will be minimised for $\cos(F_+ - F_-) = -1$, giving us the minimum

$$R = \sqrt{A_+^2 + A_-^2 - 2A_+A_-} = \sqrt{(A_+ - A_-)^2} = |A_+ - A_-|$$

Appendix B: Parameter settings for the simulations

The simulations were run with a system of base units:

1. Time: microsecond μs
2. Distance: micrometer μm
3. Mass: unit u
4. Charge: elementary charge e

The values presented in Table I assume this unit system.

TABLE I. The used settings for all the simulations

Variable	**Single particle**	**Two particles**	**100 particles**
B_0	9.65e1	9.65e1	9.65e1
V_0	2.41e6	2.41e6	2.41e6$(1 + f\cos(\omega_V t)$
d	500	500	500
Amplitude f	0, static V	0, static V	0.1 or 0.4 or 0.7 (0.7 for detailed)
Frequency ω_V	0, static V	0, static V	0.2 - 2.5 initial, 0.55-0.75 detailed
Particle mass	40.08	40.08	40.08
Particle charge	1	1	1
2***Particle 1**	r = (20, 0, 20)	r = (20, 0, 20)	N/A
	v = (0, 25, 0)	v = (0, 25, 0)	N/A
2***Particle 2**	N/A	r = (25, 25, 0)	N/A
	N/A	v = (0, 40, 5)	N/A
Particle 1-100	N/A	N/A	Randomly sampled r and v
Coulomb interactions	N/A	On or off	Off for initial scan On or off for detailed scan
Total duration	50	50	500
Timesteps n	4000 or 8000 \\ or 16000 or 32000	32000	40 000
Evolution method	FE or RK4	RK4	RK4